A Pilot Study on Secure Code Generation with ChatGPT for Web Applications

Mahesh Jamdade
University of Massachusetts Dartmouth
Dartmouth, Massachusetts, USA
mjamdade@umassd.edu

Yi Liu
University of Massachusetts Dartmouth
Dartmouth, Massachusetts, USA
yliu11@umassd.edu

ABSTRACT

Conversational Large Language Models (LLMs), such as ChatGPT, have demonstrated their potent capabilities in natural language processing tasks. This paper presents a pilot study that uses ChatGPT for generating web application code with a specific emphasis on mitigating four prevalent web application vulnerability types: SQL Injection, Cross Site Scripting, Carriage Return Line Feed Injection, and Exposure of Sensitive Information. The paper uses a case study to illustrate how the vulnerabilities in the code are mitigated with the prompts and the subsequent refinements. The study's findings summarize the security concerns in the code generated by ChatGPT, and the paper proposes a prompt pattern designed to help mitigating the potential vulnerabilities.

CCS CONCEPTS

• **Security and privacy → Web application security**.

KEYWORDS

Secure coding, web application vulnerabilities, generative AI, ChatGPT

ACM Reference Format:
Mahesh Jamdade and Yi Liu. 2024. A Pilot Study on Secure Code Generation with ChatGPT for Web Applications. In *2024 ACM Southeast Conference (ACMSE 2024), April 18–20, 2024, Marietta, GA, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3603287.3651194

1 INTRODUCTION

In recent years, Large Language Models (LLMs) have gained significant attention due to their wide usage in natural language processing (NLP) tasks, such as translation, sentiment analysis, content generation, chatbots, and many more. One of the most popular LLM-based artificial intelligence (AI) chatbots is ChatGPT, developed by OpenAI and released to the public in November 2022 [5].

Powered by GPT-3.5 and now the GPT-4 models, ChatGPT has presented its versatile capabilities not only in tasks such as answering questions, language translation, and creative writing but also in the domain of code generation.

ACMSE 2024, April 18–20, 2024, Marietta, GA, USA

ACM ISBN 979-8-4007-0237-2/24/04…$15.00
https://doi.org/10.1145/3603287.3651194

Security is an essential nonfunctional requirement of a successful software system. For web applications, neglecting security measures can expose vulnerabilities that may lead to significant risks, such as unauthorized access, data breaches, to compromise the reliability of the web application. The goal of this study is to explore ChatGPT's ability in generating secure code for web applications. It answers the following research questions:

- R1: Does ChatGPT generate code containing common web application vulnerabilities?
- R2: Are there any prompt patterns that can assist in generating secure code to mitigate the common web application vulnerabilities?

In this pilot study, we target to address four of the most common web application vulnerability types, including three injection types: *SQL Injection (SQLi)* (CWE-89), *Cross Site Scripting (XSS)* (CWE-79), *Carriage Return Line Feed (CRLF) Injection* (CWE-93), and *Exposure of Sensitive Information* (CWE-200). The selection of vulnerability types is based on the combination of OWASP's top ten [19], annual reports from cybersecurity companies [2, 20], and National Vulnerability Database (NVD) data analysis [3]. *Exposure of Sensitive Information* is a prominent web application vulnerability found in NVD [3], categorized as a vulnerability type under OWASP's No. 1 security risk "Broken Access Control" [17]. SQLi, XSS and CRLF are identified as the most critical vulnerabilities in the tested web applications [2, 20]. SQLi and XSS are leading web application vulnerabilities found in NVD [3]. These three types fall under OWASP's No. 3 security risk category "Injection" [18].

The rest of the paper is organized as follows. Section 2 introduces the four common web application vulnerability types addresses in this paper, and the web application developed for the case study. Section 3 presents the experiments conducted for security coding with ChatGPT. The refinement of prompts is applied to mitigate vulnerabilities in the implementation of backend microservices for the library system. Section 4 answers the two research questions and discusses the limitations of this pilot study. Section 5 summarizes the work and outlines the future research direction.

2 BACKGROUND

2.1 Web Application Vulnerability Types

Common Weakness Enumerations (CWEs) are a community-developed list categorizing, and describing common software and hardware security weaknesses (vulnerability types). Each CWE entry includes a unique identifier, detailed description, and potential mitigations to enhance security practices. The four vulnerability types focused on in this study are all defined with CWEs.

2.1.1 SQL Injection (SQLi). SQL Injection (SQLi), identified by the Common Weakness Enumeration (CWE) entry CWE-89, is formally described as the "Improper Neutralization of Special Elements used in an SQL Command (SQL Injection)" [13]. This vulnerability occurs when a software application improperly handles use inputs in SQL queries, allowing attackers to inject malicious SQL code into input fields. By manipulating the query's logic, attackers can gain unauthorized access or manipulate the data in the database. The common prevention and mitigation strategies for SQLi include input validation and sanitization, the use of prepared statements [6] for constructing SQL queries, Stored Procedures [22], and more.

2.1.2 Cross Site Scripting (XSS). Identified as CWE-79 "Improper Neutralization of Input During Web Page Generation (Cross-site Scripting)" [12], Cross-site scripting (XSS) allows an attacker to inject malicious code in the form of browser-side script into web pages viewed by other users [10]. Such attacks can lead to the theft of sensitive information, such as login credentials or session tokens, or performing actions on behalf of the user without their consent. The common prevention and mitigation strategies for XSS include input validation and sanitization, implementation of *Content Security Policy* [4], using *HTTPOnly cookie* flag [1], *SameSite cookie parameter* [8], and more.

2.1.3 Exposure of Sensitive Information. The Common Weakness Enumeration (CWE) identifier CWE-200, "Exposure of Sensitive Information to an Unauthorized Actor," occurs when unauthorized users gain access to confidential or private data, potentially resulting in data breaches [11]. The causes of this vulnerability can range from inadequate access controls, improper handling of sensitive information, to insufficient encryption practices. The common prevention and mitigation strategies include encryption of sensitive data, implementing strict access controls, access control assessments, regular security audits, and more [16].

2.1.4 Carriage Return Line Feed (CRLF) Injection. Identified as "CWE-93 Improper Neutralization of CRLF Sequences (CRLF Injection)," CRLF Injection occurs when an attacker is able to insert CRLF characters (`"\r\n"`) into an application through input fields or parameters [14]. These characters represent the end of a line. Attackers can exploit applications vulnerable to CRLF Injection to add new HTTP headers or manipulate existing ones, resulting HTTP response splitting. This can be combined with other attacks, such as XSS, to perform various malicious activities such as phishing attacks. The prevention and mitigation strategies for CRLF Injection typically include input validation, output encoding, and the removal of newline characters from user input to prevent the unauthorized injection of CRLF sequences [21].

2.2 Case Study

The case study used in this paper is a full-stack web application, entirely developed by using prompts fed to ChatGPT, from the requirements analysis and design to the implementation. The web application is designed for a university library that provides online access to students, staff, and library personnel. Staff and students are able to log in, search for books, renew checked-out items, and access their library records. Staff members have the additional privilege of request books. Librarians will be able to log in, add books, register new users, and search for books. We refer to this application as the "library system" in the rest of the paper.

The backend of this application is designed using Microservice architecture with 7 Microservices: *User Service*, which handles user registration, user authentication processes, profile management; *Book Service*, which manages book-related operations, including adding new books, searching for books, retrieving of book information, and handling the availability status of book; *Checkout Service*, which is responsible for book checkout and renewal processes, along with sending notifications for due dates; *Request Service*, which handles the process of requesting books, stores and retrieves book requests, and facilitates the review and approval of book requests; *Notification Service*, which manages the generation and delivery of notifications, such as sending email notifications, and in-app notifications; *Librarian Service*, which handles librarian-specific actions, such as adding and updating books in the library collection; and *Report Service*, which is responsible for generating various reports, and storing and retrieving report data. The microservices are developed using *Node.js express* and *TypeScript*, and a *PostgreSQL* database for each service.

2.3 Related Work

Khan et al. [7] have conducted a study evaluating ChatGPT's code generation capabilities using 131 code-generation prompts and 262 code samples. The findings indicate that ChatGPT has strength in generating efficient code with advanced constructs, but it exhibits limitations in visual-graphical challenges. Khoury et al. [9] have evaluated 21 programs in 5 different programming languages generated by ChatGPT for vulnerabilities. The study finds that, although ChatGPT refuses to create attack code, it generates code with vulnerabilities. In certain cases of the generated vulnerable code, ChatGPT provides advice on how to enhance its security but is not able to create a more secure version of the code. In their study [15], Nair et al. have observed that in hardware code generation, ChatGPT generates vulnerable code if not prompted carefully and then explored the strategies to adjust prompts for secure hardware code generation.

3 SECURITY CODING WITH CHATGPT

This section presents the findings of security coding with ChatGPT in the implementation of back-end microservices for the library system. While certain vulnerabilities can be mitigated in the front-end using input validation, our focus in this paper is on the back-end solutions. The examples provided in this section serve as representative cases for addressing the four vulnerability types. These vulnerability types are addressed throughout the development, not limited to the cases presented here. The code presented uses Node.js.

3.1 Addressing SQLi

When implementing the *User Service*, we fed the following prompt to ChatGPT for the implementation of methods of finding user by user name and finding user by user ID.

Initial prompt: Implement separate methods to find user by user name and id in User Service.

Figure 1 shows the implementation of the method for finding a user by username, generated by ChatGPT. In this code, the variable `username` is directly passed into the `SELECT` statement to query the database table `users` without proper sanitization. The `username` will be an input from a client through a front-end user interface. Thus, if a client enters `1' = '1` as the `username`, the `getUserByUsername` method will form the `query` as

`SELECT * FROM users WHERE username = '1' = '1';`

Since `'1' = '1'` always evaluates to true, the `WHERE` clause is true, resulting in the bypass of the authorization logic that checks for an existing username.

```
class UserService {
     static async getUserByUsername(username: string):
                       Promise<User | null> {
         const query =
            `SELECT * FROM users WHERE username = '${username}'`;
         const result = await pool.query(query);
         if (result.rows.length > 0) {
             const row = result.rows[0];
             return new User(row.id, row.username, row.email);
         } else {
             return null;
         }
     }
     ...
}
```

Figure 1: Code Segment with SQLi Vulnerability

In order to mitigate the SQLi vulnerability in the code, we fed ChatGPT the following prompt:

Security Prompt: Update the methods to secure them from SQL injection.

ChatGPT updated the code and Figure 2 shows the updated `getUserByUsername` method. The updated code applies the prepared statements [10] to mitigate the SQLi vulnerability. Instead of directly appending user input `username` to the SQL query, the prepared statement uses a placeholder (`$1` in Figure 2) for the parameter. The placeholder is securely bound to the input value `username` before the query is executed. This substitution treats the input as data instead of executable SQL code. For example, if the user inputs `1 = 1`, the prepared statement will treat it as data instead of an expression to evaluate in the SQL logic.

One of the functionalities provided by the *Book Service* is for staff members to submit the request of books by entering a book's title, its author name, and the reason of requesting it. We fed ChatGPT

```
static async getUserByUsername(username: string):
           Promise<User | null> {
     const query = 'SELECT * FROM users WHERE username = $1';
     const result = await pool.query(query, [username]);
     if (result.rows.length > 0) {
         const row = result.rows[0];
     return
         new User(row.id, row.username, row.email);
     } else {
          return null;
     }
}
```

Figure 2: Code Segment with SQLi Vulnerability Mitigated

```
class RequestService {
    static async submitRequest(user_id: number,
          book_title: string,   book_author: string,
          justification: string): Promise<Request> {
      const result = await pool.query(
            'INSERT INTO requests
                (user_id,book_title,book_author,justification)
             VALUES ($1, $2, $3, $4) RETURNING *',
           [user_id,book_title,book_author,justification]
          );
      return result.rows[0];
    }
}
```

Figure 3: Code Segment with XSS Vulnerability

the following prompt to implement method `submitRequest` in class `RequestService`.

Initial prompt: Implement RequestService.submitRequest and use all the properties as per the database schema.

Figure 3 illustrates the code segment generated by ChatGPT. The values of `book_title`, `book_author`, and `justification` are collected from user's inputs in the front-end user interface and passed to the *Book Service*. The method `submitRequest` uses prepared statements to address the SQLi vulnerability by binding input values `user_id`, `book_title`, `book_author`, and `justification` for the parameters `$1`, `$2`, `$3`, and `$4`, respectively. However, the code is vulnerable to XSS. For instance, an attacker can enter the script shown below as the user input for `justification`.

```
 <iframe src="https://some_audio_track"
allow="autoplay; encrypted-media; picture-in-picture">
</iframe>
```

A new record will be saved to table `requests` with this malicious script in the field `justification`. In the future, when this book request is loaded for review by a librarian, its justification will be a soundtrack specified by the "src" field in malicious script.

We asked ChatGPT to mitigate the XSS vulnerability with the following prompt:

Security prompt: Modify the code to handle XSS vulnerabilty.

ChatGPT added a method `escape` for escaping the characters < and < and then applied it for sanitizing the inputs of `book_title`, `book_author`, and `justification` in method `submitRequest`. Figure 4 illustrates the implementation.

With sanitization, the script for XSS attack shown above becomes

`<iframe ... ></iframe>`

which no longer contains the markups for loading a soundtrack. Thus, the XSS vulnerability in this case is mitigated. ChatGPT comments that the implemented `escape` method is a simplified version, which only addresses < and >. Other characters, such as the unescaped '&' character, still pose a risk since they can be used to create HTML entities, leading to XSS attacks.

3.2 Addressing Exposure of Sensitive Information

The *Notification Service* has a functionality for sending email notifications to clients. One of the classes ChatGPT generated for this functionality is `EmailUtils`. The initially generated code did not

```
class RequestService {
     static escape(input: string): string {
       // For simplicity, a basic implementation is shown here
       // (not recommended for production)
       return input.replace(/</g,'&lt;').replace(/>/g,'&gt;');
     }

     static async submitRequest(user_id: number,
          book_title: string, book_author: string,
          justification: string):Promise<Request> {

        // Sanitize user inputs to prevent XSS
        const sanitizedTitle = this.escape(book_title);
        const sanitizedAuthor = this.escape(book_author);
        const sanitizedJustification =
                                  this.escape(justification);

        // Insert sanitized inputs into the database
        const result = await pool.query(
            'INSERT INTO requests
                (user_id,book_title,book_author,justification)
             VALUES ($1, $2, $3, $4) RETURNING *',
            [user_id, sanitizedTitle, sanitizedAuthor,
             sanitizedJustification]);

        return result.rows[0];
    }
}
```

Figure 4: Code Segment with XSS Vulnerability Mitigated

use email templates, so we requested ChatGPT to modify the class to include the email template using the prompt below.

Initial prompt: Modify EmailUtils to include the email template.

Figure 5 shows the partial code of `EmailUtil` that ChatGPT generated. The email address and password are specifically stored in the `auth` field which contains authentication details. Storing sensitive information in source code makes it to be accessible to anyone who can access the code, thus it introduces the Exposure of Sensitive Information vulnerability.

We fed the ChatGPT the following prompt to address the Exposure of Sensitive Information vulnerability and other possible vulnerabilities in the code.

Security prompt: Modify the code to ensure it does not expose any sensitive information and is not vulnerable to any type of attacks

ChatGPT enhanced the security by using the environment variables to store the credentials so that the sensitive information is not explicitly given in the code. In the updated code, user's email address and password are now stored in `env.EMAIL_USER` and `env.EMAIL_PASSWORD`, respectively. The Exposure of Sensitive Information vulnerability is mitigated.

3.3 Addressing Carriage Return Line Feed (CRLF) Injection

While ChatGPT successfully mitigated the Exposure of Sensitive Information vulnerability in the updated `sendEmail` method, the code in Figure 6 still has other types of vulnerabilities, even though we asked it to address "other possible vulnerabilities in the code."

The code is vulnerable to Carriage Return Line Feed (CRLF) injection, which can be exploited in the `mailOptions` object. An attacker can inject data containing CRLF characters (`\r\n`) to the

```
import nodemailer from 'nodemailer';
class EmailUtils {
    static async sendEmail(to: string, subject: string,
           body: string): Promise<void> {
        try {
           // ConListing nodemailer transporter
           // (you may need to provide your own SMTP details)
           const transporter = nodemailer.createTransport({
                service: 'gmail',
                auth: {
                      // replace with your email
                      user: 'your-email@gmail.com',
                      // replace with your password
                      pass: 'your-password',
                },
            });

           // Email options
           const mailOptions = {
               from: 'your-email@gmail.com',
               to,
               subject,
               text: body,
           };

           // Send email
           await transporter.sendMail(mailOptions);
       }catch (error) {
           console.error('Error sending email:', error);
           throw new Error('Error sending email');
       }
    }
    . . .
}
```

Figure 5: Code Segment with Exposure of Sensitive Information Vulnerability

```
import nodemailer from 'nodemailer';
class EmailUtils {
    static async sendEmail(to: string, subject: string,
           body: string): Promise<void> {
       try {
           //Use environment variables to store sensitive info
           const transporter = nodemailer.createTransport({
                service: 'gmail',
                auth: {
                    user: process.env.EMAIL_USER,
                    pass: process.env.EMAIL_PASSWORD,
                },
           });

           const mailOptions = {
                from: process.env.EMAIL_USER,
                to,
                subject,
                text: body,
           };

           // Send email
           await transporter.sendMail(mailOptions);
       }catch (error) {
           console.error('Error sending email:', error);
           throw new Error('Error sending email');
       }
    }
    . . .
}
```

Figure 6: Code Segment with Exposure of Sensitive Information Vulnerability Mitigated

```
// Sanitize input to prevent CRLF injection
// Remove any newline characters
to = to.replace(/[\r\n]+/g, '');
subject = subject.replace(/[\r\n]+/g, );
body = body.replace(/[\r\n]+/g, '');
// Email options
const mailOptions = {
    from: process.env.EMAIL_USER,
    to,
    subject,
    text: body,
};
```

Figure 7: Code Segment with CRLF Vulnerability Mitigated

`to`, `subject`, and `text` fields. For example, the attacker can pass the following to the `to` field:

`malicious@example.com\r\nCC:victim@example.com`

This could lead to CRLF injection, allowing the attacker to add a new header `CC: victim@example.com`, thereby adding a CC recipient (`victim@example.com`) to the email.

We fed ChatGPT the following prompt and it updated the code by sanitizing the inputs of `to`, `subject`, and `body` with newline characters `\r\n` removed using regular expressions. Figure 7 presents the added statements for sanitization.

Security prompt: Modify this method to prevent against CRLF vulnerability.

With the sanitization, the previous example of attack will not succeed because the input string will be sanitized to

`malicious@example.comCC:victim@example.com`.

This sanitized input will not add a CC recipient (`victim@example.com`) to the email.

4 DISCUSSION

Our development of the library system using ChatGPT serves as the basis for answering the research questions raised in Section 1.

4.1 R1: Does ChatGPT Generate Code Containing Common Web Application Vulnerabilities?

In our experiments using ChatGPT to develop the library system, we observed that ChatGPT has generated code containing common web application vulnerabilities. We found that the generated code contained all four types of vulnerabilities focused in this paper. Figures 1, 3, and 5 represent those cases. Similar vulnerable code can be found elsewhere in the application.

ChatGPT does not inherently address vulnerabilities in code unless prompted to do so. We noticed that the vulnerability type needs to be explicitly specified in the prompts; otherwise, ChatGPT may not identify which vulnerability type to address if a similar case has not been handled before. As demonstrated in the "initial prompts" in Section 3, ChatGPT may not naturally address those vulnerabilities. In another case in Section 3.3, when we asked ChatGPT to "modify the code to ensure it does not expose any sensitive information and is not vulnerable to any type of attacks", ChatGPT addressed the specified exposure sensitive information vulnerability but did not address the CRLF Injection vulnerability in the code. This was because the prompt did not explicitly instruct it, and no similar cases were handled before in the same chat.

We also observed that if we requested ChatGPT to mitigate the code to address a specific vulnerability type, ChatGPT would address it in the subsequent similar coding requests, even without explicitly mentioning that vulnerability type in the prompt. This implies that, in the same chat, if ChatGPT becomes aware of a vulnerability type from specific prompts designed to address it, it will address the same vulnerability type in later, similar coding scenarios. An example illustrating this scenario is in Figure 3. Although we did not explicitly request ChatGPT to address SQLi in the prompt for implementing `RequestService.submitReques`, ChatGPT did address SQLi in the code. The reason is that a security prompt instructing ChatGPT to update methods to mitigate SQLi (in Section 3.1) was given earlier in the same chat. ChatGPT addressed the SQLi vulnerability by applying prepared statements to handle input values when constructing the query for the database. The method shown in Figure 3 follows a similar pattern, which takes input values to construct a query.

4.2 R2: Are there any Prompt Patterns that can Assist in Generating Secure Code to Mitigate the Common Web Application Vulnerabilities?

As discussed in R1, ChatGPT requires specific instructions on the vulnerability type to address. The user of ChatGPT needs to be aware of the potential vulnerability type(s) in the implementation to provide instructions to ChatGPT.

Based on our experiments, the following prompt pattern can be used to prevent or mitigate a certain vulnerability type.

Security prompt: Implement [functionality description] | Mitigate [specific code] to ensure it does not have [vulnerability type(s)].

The security prompt describes two options: either implement a functionality as described in "functionality description" or mitigate the existing code as described in "specific code", aiming to address potential "vulnerability types" in the code to be generated.

4.3 Limitations of our Work

In this study, we focused on refining prompts to enhance the security implemented by ChatGPT. ChatGPT typically provided one mitigation solution to address a particular vulnerability type. However, multiple solutions could be applied to address a vulnerability type for multi-layered protection. For example, when addressing the SQLi vulnerability, input sanitization along with prepared statements can be applied to enhance the coding security. In this pilot study, we did not further refine the prompts to request the implementation of more complete mitigation solutions. In some cases, ChatGPT provided simplified solutions. For example, in dealing with XSS, it offered an escape method that only handled < and >. For a complete solution, it is necessary to consider additional characters. In this study, we did not refine the prompts to request more complete solutions.

Our study focuses on addressing four of the most common web application vulnerabilities: *SQLi*, *XSS*, *CRLF Injection*, and *Exposure of Sensitive Information*. While these vulnerabilities are significant,

exploring the broader landscape of web application vulnerability types with ChatGPT is worthwhile. Other notable vulnerability types include *Cross-Site Request Forgery (CSRF)*(CWE-352) and *Buffer Overflow* (CWE-120), and more.

5 CONCLUSION

This pilot study explores the application of ChatGPT for generating secure web application code, with a specific focus on addressing four of the most common web application vulnerabilities, SQLi, XSS, CRLF Injection, and Exposure of Sensitive Information. The findings indicate that ChatGPT does not naturally address vulnerabilities in code without specific instructions. Through empirical experiments, the study proposes a prompt pattern to help mitigate potential vulnerabilities in code.

The future work will be focused on two directions: firstly, expanding vulnerability coverage to include Buffer Overflow, Cross-Site Request Forgery, Unrestricted Upload of File, and Path Traversal, designing prompts to include multiple mitigation strategies in generated code, and documenting them with formal pattern structures [23]; secondly, experimenting with secure code generation using alternative LLMs such as OpenAI Codex, and comparing results with ChatGPT.

REFERENCES

[1] Jianjun Chen, Jian Jiang, Haixin Duan, Tao Wan, Shuo Chen, Vern Paxson, and Min Yang. 2018. We still Don't Have Secure Cross-Domain Requests: an Empirical Study of CORS. In *27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, USA, 1079–1093.

[2] Edgescan. 2023. *2023 Vulnerability Statistics Report.* https://www.edgescan.com/intel-hub/stats-report

[3] Onyeka Ezenwoye, Yi Liu, and Willam Patten. 2020. Classifying Common Security Vulnerabilities by Software Type. In *Proceedings of the 32nd International Conference on Software Engineering and Knowledge Engineering (SEKE 2020)*. Pittsburgh, USA, 61–64.

[4] Gertjan Franken, Tom Van Goethem, Lieven Desmet, and Wouter Joosen. 2023. A Bug's Life: Analyzing the Lifecycle and Mitigation Process of Content Security Policy Bugs. In *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, USA, 3673–3690.

[5] Kristi Hines. 2023. History Of ChatGPT: A Timeline Of The Meteoric Rise Of Generative AI Chatbots. https://www.searchenginejournal.com/history-of-chatgpt-timeline/488370/

[6] Etienne Janot and Pavol Zavarsky. 2008. Preventing SQL Injections in Online Applications: Study, Recommendations and Java Solution Prototype Based on the SQL DOM. In *OWASP Application Security Conference*.

[7] Muhammad Fawad Akbar Khan, Max Ramsdell, Erik Falor, and Hamid Karimi. 2023. Assessing the Promise and Pitfalls of ChatGPT for Automated Code Generation. *arXiv preprint* (2023). https://doi.org/arXiv:2311.02640

[8] Soheil Khodayari and Giancarlo Pellegrino. 2022. The State of the SameSite: Studying the Usage, Effectiveness, and Adequacy of SameSite Cookies. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, San Francisco, USA.

[9] Raphaël Khoury, Anderson R Avila, Jacob Brunelle, and Baba Mamadou Camara. 2023. How Secure is Code Generated by ChatGPT? *arXiv preprint arXiv:2304.09655* (2023).

[10] Miao Liu, Boyu Zhang, Wenbin Chen, and Xunlai Zhang. 2019. A Survey of Exploitation and Detection Methods of XSS Vulnerabilities. *IEEE Access* 7 (2019). https://doi.org/10.1109/ACCESS.2019.2960449

[11] MITRE. 2006-2023. *CWE-200: Exposure of Sensitive Information to an Unauthorized Actor.* https://cwe.mitre.org/data/definitions/200.html

[12] MITRE. 2006-2023. *CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting').* https://cwe.mitre.org/data/definitions/79.html

[13] MITRE. 2006-2023. *CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection').* https://cwe.mitre.org/data/definitions/89.html

[14] MITRE. 2006-2023. *CWE-93: Improper Neutralization of CRLF Sequences ('CRLF Injection').* https://cwe.mitre.org/data/definitions/93.html

[15] Madhav Nair, Rajat Sadhukhan, and Debdeep Mukhopadhyay. 2023. Generating Secure Hardware Using ChatGPT Resistant to CWEs. *Cryptology ePrint Archive* (2023). https://eprint.iacr.org/2023/212.pdf

[16] OWASP. 2017. *OWASP Top Ten 2017: OWASP A3:2017-Sensitive Data Exposure.* https://owasp.org/www-project-top-ten/2017/A3_2017-Sensitive_Data_Exposure

[17] OWASP. 2021. *OWASP Top 10 A01:2021 – Broken Access Control.* https://owasp.org/Top10/A01_2021-Broken_Access_Control/

[18] OWASP. 2021. *OWASP Top 10 A03:2021 – Injection.* https://owasp.org/Top10/A03_2021-Injection/

[19] OWASP. 2021. *OWASP Top Ten Web Application Security Risks.* https://owasp.org/www-project-top-ten/

[20] Veracode. 2023. *State of Software Security 2023: Annual Report on the State of Application Security.* https://www.veracode.com/state-of-software-security-report

[21] Wallam. 2022. *CRLF Injection Attack: Examples and Prevention.* https://www.wallarm.com/what/crlf-injection-attack

[22] Kei Wei, Muthusrinivasan Muthuprasanna, and Suraj Kothari. 2006. Preventing SQL Injection Attacks in Stored Procedures. In *Australian Software Engineering Conference (ASWEC'06)*. IEEE, Sydney, Australia.

[23] Jules White, Sam Hays, Quchen Fu, Jesse Spencer-Smith, and Douglas C Schmidt. 2023. Chatgpt Prompt Patterns for Improving Code Quality, Refactoring, Requirements Elicitation, and Software Design. *arXiv preprint arXiv:2303.07839* (2023).